Белорусский государственный университет
Факультет прикладной математики и информатики
Кафедра информационных технологий и компьютерной графики

Лабораторная работа №2

«Обработка изображений: глобальная пороговая обработка, фильтры порядковых статистик и сжатие по методу Хаффмана»

Вариант №10

Выполнил:
студент 3 курса
группы 4
Лемешевский Арсений

Минск, 2026

1. Введение и задачи работы

Цель работы: исследование методов глобальной пороговой сегментации и нелинейной фильтрации на основе порядковых статистик (вариант №10), изучение алгоритма сжатия графической информации без потерь по методу Хаффмана и реализация этих алгоритмов в едином desktop-приложении.

Поставленные задачи:

1. Изучить и реализовать вручную (без OpenCV и иных готовых библиотек обработки изображений) глобальную пороговую обработку: базовый итеративный алгоритм, метод Оцу и ручной выбор порога $T \in [0, 255]$.
2. Реализовать фильтры порядковых статистик (медиана, минимум, максимум, средняя точка) с настраиваемым размером окна (3×3, 5×5, 7×7), формой апертуры (квадрат, крест) и тремя способами обработки границ.
3. Реализовать кодер и декодер Хаффмана без использования zlib, libpng и других компрессоров: построение дерева, побитовую упаковку, собственный формат файла `.huff` и проверку отсутствия потерь.
4. Разработать Qt-приложение с разделением ответственности: логика обработки и кодирования не зависит от интерфейса.
5. Провести эксперименты на изображениях с разными свойствами и вычислить метрики сжатия: коэффициент сжатия, экономию места, энтропию H и среднюю длину кода L_{avg}.

2. Стек технологий и исходный код

Репозиторий проекта: https://github.com/ArseniyLem/Computer-Graphics/tree/main/lab%202

Приложение выполнено как настольное Qt-приложение на C++:

- **C++20** — язык реализации; шаблоны, `std::array`, `std::priority_queue`, `std::nth_element`, структурная типизация данных без ручного управления памятью.
- **Qt 6 (QtWidgets)** — окна и элементы интерфейса: `QTabWidget`, `QScrollArea`, `QSlider`, `QSpinBox`, `QComboBox`, `QRadioButton`; `QImage` — контейнер пикселей; `QElapsedTimer` — замер времени операций.
- **CMake ≥ 3.21** и **Qt Creator** — сборка и разработка; исходники в кодировке UTF-8.
- Ограничение задания выдержано: алгоритмы пороговой обработки, фильтрации и кодирования написаны вручную; сторонние библиотеки (OpenCV, zlib, libpng) в алгоритмической части не используются. Qt применяется только как оболочка интерфейса, для чтения исходного файла и вывода изображения на экран.

3. Математические основы

3.1. Глобальная пороговая обработка

Цветное изображение приводится к градациям серого по формуле ITU-R BT.601 (в коде — в целочисленной арифметике с округлением):

$$Y = 0{,}299 \cdot R + 0{,}587 \cdot G + 0{,}114 \cdot B, \quad Y = \lfloor (299 \cdot R + 587 \cdot G + 114 \cdot B + 500) / 1000 \rfloor$$

Бинаризация с глобальным порогом T выполняется по правилу «объект ярче фона»:

$$g(x, y) = 255, \text{ если } f(x, y) > T; \quad g(x, y) = 0, \text{ иначе}$$

На практике преобразование реализовано через таблицу подстановки из 256 элементов, поэтому условие проверяется не для каждого пикселя, а один раз для каждой градации яркости.

Итеративный алгоритм (Basic Global Thresholding). Все вычисления ведутся по гистограмме яркости h(i), i = 0…255, поэтому одна итерация стоит O(256), а не O(W·H):

1. Начальное приближение — средняя яркость изображения: $T_0 = \Sigma\, i \cdot h(i) / \Sigma\, h(i)$.
2. Пиксели делятся на классы $G_1 = \{f \le T\}$ (фон) и $G_2 = \{f > T\}$ (объект); вычисляются средние μ_1 и μ_2.
3. Новое значение порога: $T_{k+1} = (\mu_1 + \mu_2) / 2$.
4. Шаги 2–3 повторяются, пока $|T_{k+1} - T_k| \ge \varepsilon$, где $\varepsilon = 0{,}5$. Если один из классов оказался пустым (например, изображение однотонное), итерации прекращаются; число итераций ограничено 256.

Метод Оцу. Порог выбирается так, чтобы максимизировать межклассовую дисперсию. Пусть $p_i = h(i)/N$ — доля пикселей с яркостью i, тогда для порога t:

$$\omega_0(t) = \Sigma_{i \le t}\, p_i, \quad \omega_1(t) = 1 - \omega_0(t)$$

$$\mu_0(t) = \Sigma_{i \le t}\, i \cdot p_i / \omega_0(t), \quad \mu_1(t) = \Sigma_{i > t}\, i \cdot p_i / \omega_1(t)$$

$$\sigma_B^2(t) = \omega_0(t) \cdot \omega_1(t) \cdot (\mu_0(t) - \mu_1(t))^2, \quad T^* = \operatorname{argmax}\, \sigma_B^2(t)$$

Все величины вычисляются за один проход по гистограмме накопительными суммами (сложность O(256)). Если максимум достигается на целом промежутке порогов (это бывает, когда между модами гистограммы есть «пустая» зона), возвращается середина этого промежутка.

Ручной режим. Порог T задаётся слайдером или полем ввода в диапазоне 0…255; результат обновляется сразу при перемещении слайдера, так как бинаризация линейна по числу пикселей и не требует заметного времени.

3.2. Фильтры порядковых статистик

Для каждого пикселя (x, y) рассматривается окно, заданное структурирующим элементом S размером k = 2r + 1. Значения яркости внутри окна упорядочиваются, и в качестве результата берётся заданная порядковая статистика:

$$\textit{Медиана: } g(x, y) = \mathrm{med}\{ f(x+i, y+j) \mid (i, j) \in S \}$$

$$\textit{Минимум: } g(x, y) = \min\{ \dots \}, \quad \textit{Максимум: } g(x, y) = \max\{ \dots \}$$

$$\textit{Средняя точка: } g(x, y) = (\min\{ \dots \} + \max\{ \dots \}) / 2$$

Форма апертуры: **квадрат** содержит k^2 элементов, **крест** (центральная строка и центральный столбец) — $4r + 1$ элементов. Цветные изображения фильтруются независимо по каналам R, G, B.

Области применения фильтров:

- **Медиана** — подавление импульсного шума «соль и перец» с сохранением границ.
- **Минимум** (эрозия) удаляет светлые выбросы («соль»), но расширяет тёмные; **максимум** (дилатация) — наоборот.
- **Средняя точка** — эффективна при гауссовом и равномерном шуме.

Обработка границ. Для пикселей у края часть окна выходит за изображение. Реализованы три режима, в которых значение за границей определяется так:

- **Replicate (Clamp to Edge)** — берётся ближайший крайний пиксель: $f(-1) = f(0)$.
- **Reflect** — зеркальное отражение с дублированием крайнего пикселя: $f(-1) = f(0)$, $f(-2) = f(1)$.
- **Zero Padding** — за границей яркость равна нулю.

Сложность фильтра — $O(W \cdot H \cdot n)$, где n — число элементов апертуры. Для медианы использован `std::nth_element` (частичная сортировка, в среднем $O(n)$ вместо $O(n \log n)$ для полной сортировки окна).

3.3. Сжатие по методу Хаффмана и информационные характеристики

Метод Хаффмана — префиксное кодирование переменной длины: частым байтам назначаются короткие коды, редким — длинные. Изображение рассматривается как поток байтов (строка за строкой), алфавит содержит 256 символов.

Построение кода:

1. Подсчитываются частоты f_s байтов 0…255.
2. Каждому встретившемуся байту соответствует лист; листья помещаются в очередь с приоритетом (min-heap, `std::priority_queue`).
3. Пока в очереди больше одного узла, извлекаются два узла с наименьшими частотами и объединяются в родителя с суммарной частотой.
4. Обход дерева в глубину: переход влево добавляет к коду бит 0, вправо — бит 1. Код листа — путь от корня.

Информационные характеристики. Для вероятностей $p_s = f_s / N$ и длин кодов l_s:

$$H = -\Sigma\, p_s \cdot \log_2 p_s, \quad L_{avg} = \Sigma\, p_s \cdot l_s, \quad H \le L_{avg} < H + 1$$

$$K = V_{исх} / V_{сжат}, \quad \textit{Экономия} = (1 - V_{сжат} / V_{исх}) \cdot 100\ \%$$

Энтропия H — теоретическая нижняя граница средней длины кода при кодировании символов независимо друг от друга. Поскольку любой код занимает не менее одного бита на символ, для байтового алфавита коэффициент сжатия не может превысить 8:1 — это важная оценка для интерпретации результатов эксперимента.

Битовая упаковка. Коды записываются в буфер побитовыми операциями. Накопитель сдвигается влево на длину кода и объединяется с ним операцией ИЛИ; как только накоплено 8 бит и более, готовый байт выталкивается в `std::vector<uint8_t>`:

```
acc = (acc << len) | (code & ((1 << len) - 1));
count += len;
while (count >= 8) { count -= 8; out.push_back(uint8_t(acc >> count)); }
```

Последний неполный байт дополняется нулевыми битами справа. Декодирование выполняется побитовым спуском по дереву от корня до листа; число декодируемых символов известно из заголовка, поэтому добавочные нули в конце потока не мешают.

4. Архитектура приложения

Приложение построено по принципу разделения ответственности (Separation of Concerns). Алгоритмические модули не зависят от виджетов и могут быть протестированы отдельно; главное окно содержит только отображение и вызовы методов бизнес-логики.

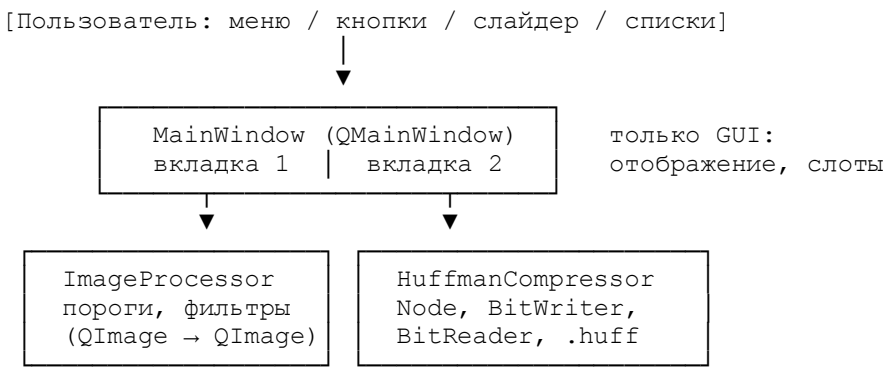


Таблица 1. Модули проекта

Файл	Назначение
`imageprocessor.h/.cpp`	Приведение к градациям серого, гистограмма, итеративный порог, метод Оцу, бинаризация, фильтры порядковых статистик.
`huffmancore.h/.cpp`	Узел дерева `Node`, `BitWriter`, `BitReader`, `HuffmanCompressor` (сжатие, распаковка, нормализация формата, сравнение изображений).
`mainwindow.h/.cpp`	Главное окно: две вкладки, меню, статус-бар, слоты, замер времени (`QElapsedTimer`).
`imageview.h`	Виджет просмотра изображения (`QScrollArea` + `QLabel`): режим «по размеру окна» и 100 % с прокруткой.

Файл	Назначение
`main.cpp`, `CMakeLists.txt`	Точка входа и описание сборки.

Хранение узлов дерева. Узлы лежат в `std::vector<Node>` и ссылаются друг на друга индексами, а не сырыми указателями, поэтому ручное управление памятью не требуется, а копирование и разрушение дерева безопасны.

Режим «цепочка». Фильтр всегда применяется к исходному изображению, результат сохраняется отдельно. Если на вкладке 1 включена опция «Порог — к результату фильтра», бинаризация применяется к отфильтрованному изображению. Это исключает повторную бинаризацию уже бинарного результата при движении слайдера.

5. Формат файла .huff

Файл содержит заголовок фиксированной длины 29 байт, таблицу частот и битовый поток. Все многобайтные поля записываются в порядке little-endian явными побайтовыми операциями, поэтому формат не зависит от архитектуры процессора.

Таблица 2. Структура заголовка

Смещение	Размер, байт	Поле	Описание
0	4	Сигнатура	Magic bytes: `H`, `U`, `F`, `F`
4	1	Версия	Версия формата (1)
5	1	Формат QImage	0 — Grayscale8, 1 — RGB32, 2 — ARGB32
6	1	Каналы	1 / 3 / 4; проверяется на соответствие формату
7	4	Ширина W	Целое без знака
11	4	Высота H	Целое без знака
15	8	Число байт данных	W·H·число каналов
23	4	Контрольная сумма	FNV-1a (32 бита) исходного потока байтов
27	2	N	Число различных байтов (1…256)
29	5·N	Таблица частот	Пары {байт: 1 байт, частота: 4 байта}, по возрастанию байта
29 + 5·N	до конца файла	Битовый поток	Коды Хаффмана, старший бит первым, добор нулями

Дерево Хаффмана в файле **не хранится**: декодер строит его по таблице частот тем же детерминированным алгоритмом. Одинаковый результат гарантируется тем, что при равных частотах из очереди извлекается узел, созданный раньше (ключ очереди — пара «частота, порядковый номер узла»). Такое решение экономит место по сравнению с сериализацией структуры дерева.

Порядок байтов в потоке не привязан к внутреннему представлению `QImage`: для RGB32 и ARGB32 кодируются каналы R, G, B (и A), считанные через `qRed`, `qGreen`, `qBlue`, `qAlpha`. Для RGB32 альфа-канал не хранится (он всегда равен 255), поэтому исходный размер данных составляет 3 байта на пиксель, а не 4.

6. Проблемы реализации и их решения

6.1. Плато функции Оцу

Суть проблемы. Если в гистограмме между двумя модами есть пустой промежуток, межклассовая дисперсия $\sigma_B^2(t)$ принимает одинаковое максимальное значение для всех порогов внутри промежутка. Простой выбор первого максимума даёт порог, прижатый к левой моде, что визуально «смещает» сегментацию.

Решение. При вычислении запоминаются первый и последний порог, на которых достигнут максимум (значения идентичны, так как накопительные суммы на пустых интервалах не меняются), и возвращается их среднее. На тестовой гистограмме с модами 60–80 и 170–190 метод Оцу дал T = 124, итеративный алгоритм — T = 125; оба значения лежат внутри пустого промежутка.

6.2. Однозначное восстановление дерева Хаффмана

Суть проблемы. Дерево Хаффмана неединственно: при равных частотах слияние можно выполнить в разном порядке и получить разные коды одинаковой средней длины. Если кодер и декодер построят разные деревья, данные не восстановятся.

Решение. Ключом очереди с приоритетом сделана пара (частота, номер узла); листья создаются в порядке возрастания значения байта, внутренние узлы — по мере слияния. Алгоритм полностью детерминирован, поэтому в файл достаточно записать таблицу частот.

6.3. Вырожденный случай — один различный байт

Суть проблемы. Если все байты изображения одинаковы (однотонная заливка), дерево состоит из одного листа, и длина его кода равна нулю: битовый поток пуст, а средняя длина кода и обход дерева теряют смысл.

Решение. Для единственного символа создаётся корень с одним левым потомком: символ получает код «0» длиной 1 бит. Кодер и декодер обрабатывают этот случай тем же кодом, что и общий, без специальных веток. Побочный эффект — для однотонного изображения $L_{avg} = 1$ при H = 0, а коэффициент сжатия ограничен значением около 8.

6.4. Границы окна фильтра

Суть проблемы. Проверка выхода за границу для каждого элемента окна внутри тройного цикла (пиксель × элемент окна × канал) заметно замедляет фильтр и усложняет код для трёх режимов обработки границ.

Решение. Перед фильтрацией канал копируется в «дополненный» массив размером (W + 2r) × (H + 2r), заполненный согласно выбранному режиму (повтор, отражение или нули). Карта столбцов вычисляется один раз и используется для всех строк. Основной цикл обращается к соседям по предвычисленным смещениям без проверок границ. Режим Reflect реализован циклическим отражением, поэтому корректен и при окне, большем половины изображения.

6.5. Выравнивание строк QImage

Суть проблемы. Строки `QImage` выровнены по 32 битам, поэтому для `Grayscale8` длина строки в памяти может превышать ширину изображения. Копирование всего буфера одним блоком даёт сдвиг пикселей и искажённые изображения.

Решение. Доступ к данным выполняется построчно через `scanLine()` / `constScanLine()` с копированием ровно W·(число каналов) байт на строку; выравнивающие байты в сжатый поток не попадают.

6.6. Защита от повреждённых файлов

Суть проблемы. Файл `.huff` с искажённым заголовком может содержать огромное значение числа байт данных. Наивный декодер выделит под результат гигабайты памяти или выйдет за границы буфера.

Решение. Перед декодированием проверяются: сигнатура и версия; код формата и число каналов; размеры W, H > 0; равенство числа байт данных произведению W·H·каналы; строгое возрастание байтов в таблице и ненулевые частоты; равенство суммы частот числу байт данных; условие «размер потока в битах ≥ числа символов» (каждый символ занимает не менее 1 бита), которое отсекает заведомо ложные заголовки до выделения памяти. Дополнительно после декодирования сверяется контрольная сумма FNV-1a. Любое нарушение приводит к исключению, которое показывается пользователю в диалоге, а не к аварийному завершению.

6.7. Накладные расходы таблицы частот

Суть проблемы. Таблица частот занимает до 5·256 = 1280 байт, поэтому заголовок достигает 1309 байт. Для малых изображений и для изображений с почти равномерным распределением байтов сжатый файл оказывается **больше** исходного (K < 1).

Решение. Это осознанный компромисс: хранение таблицы частот проще и надёжнее, чем сериализация дерева, а на практически значимых размерах (десятки килобайт и более) накладные расходы пренебрежимо малы. Для таких случаев в интерфейсе отдельно показываются размеры заголовка и битового потока, а в отчёте результаты K < 1 интерпретируются как отсутствие избыточности в данных.

7. Реализация пользовательского интерфейса

Интерфейс построен на `QTabWidget` и состоит из двух вкладок; общие операции (открытие изображения, сохранение результата, режим «по размеру окна») вынесены в меню. В строке состояния отображаются выполненная операция, её параметры и время в миллисекундах.

Таблица 3. Элементы интерфейса

Элемент	Тип	Назначение
Вкладка 1. Просмотр	`QScrollArea` + `QLabel`	Side-by-Side: «Исходное» и «Обработанное»; синхронная прокрутка в режиме 100 %.
Метод порога	3 × `QRadioButton`	Итеративный / Оцу / Ручной; при смене метода результат обновляется сразу.
Порог T	`QSlider` + `QSpinBox`	Значения 0…255; доступны в ручном режиме, в автоматических — отображают вычисленный порог.
Гистограмма	`QLabel` с `QPixmap`	Гистограмма яркости (логарифмическая шкала) с красной линией порога T.
Цепочка	`QCheckBox`	Применять порог к результату фильтрации.
Фильтр	`QComboBox`	Медиана / Минимум / Максимум / Средняя точка.
Окно и границы	3 × `QComboBox`	Размер 3×3, 5×5, 7×7; форма «квадрат» или «крест»; Clamp / Reflect / Zero.
Вкладка 2. Просмотр	`QScrollArea` + `QLabel`	Исходное изображение и восстановленное из `.huff`.
«Сжать в .huff…»	`QPushButton`	Сжатие, запись файла и немедленное декодирование из памяти для проверки.
«Распаковать .huff…»	`QPushButton`	Чтение файла, декодирование, сравнение с загруженным оригиналом.
Метрики	`QFormLayout` из `QLabel`	Размеры в байтах и КБ, K, экономия, H, L_{avg}, время сжатия и декодирования, результат Lossless-проверки.

Размеры сжатых данных сравниваются с размером **несжатых** пикселей (W·H·число каналов), а не с размером файла-источника: PNG и JPEG уже сжаты, поэтому корректное сравнение возможно только с «сырым» изображением. Размер исходного файла на диске выводится отдельной строкой справочно. Время сжатия и декодирования измеряется `QElapsedTimer` только вокруг алгоритма, без учёта операций с диском.

8. Тестовая база

Таблица 4. Рекомендуемый тестовый набор

Изображение	Цель	Ожидаемый результат
Фото или текст с шумом «соль и перец» (5–30 %)	Медиана против импульсного шума	Медиана 3×3 убирает почти весь шум; минимум удаляет «соль», но утолщает «перец», максимум — наоборот.
Фото с гауссовым шумом	Средняя точка и медиана	Средняя точка работает лучше, чем на импульсном шуме.
Скан страницы, монета на фоне (бимодальная гистограмма)	Метод Оцу и итеративный алгоритм	Близкие значения порога; чёткое разделение объекта и фона.
Малоконтрастный снимок (диапазон яркости ≈ 80…140)	Устойчивость автоматических порогов	Фиксированный T = 128 бесполезен; Оцу находит порог внутри узкого диапазона.
Снимок с неравномерной освещённостью	Предел применимости глобального порога	Результат плохой на затенённых областях — иллюстрация необходимости адаптивных методов.
Шумный текст: медиана, затем Оцу	Режим «цепочка»	Бинаризация заметно чище, чем без предварительной фильтрации.
Логотип в 2–4 цвета, чертёж, чёрно-белый скан	Лучший случай сжатия	H ≈ 0,5…2 бит, K = 4…8.
Сильно затемнённое фото, иллюстрация с плавными областями	Средний случай	H ≈ 4…6 бит, K ≈ 1,3…2.
Детализированное фото, шум, градиент по всем 256 уровням	Худший случай сжатия	H ≈ 7,5…8 бит, K ≈ 1 и менее.

9. Выводы

В ходе выполнения лабораторной работы:

1. Изучены и реализованы вручную два метода автоматического выбора глобального порога — итеративный и метод Оцу — и ручной режим; показано, что метод Оцу корректно обрабатывает плато максимума межклассовой дисперсии.
2. Реализованы фильтры порядковых статистик (медиана, минимум, максимум, средняя точка) с окнами 3×3, 5×5, 7×7, апертурами «квадрат» и «крест» и тремя режимами обработки границ; корректность подтверждена сравнением с независимой реализацией на 72 комбинациях параметров.
3. Разработан кодер и декодер Хаффмана без сторонних компрессоров: дерево на `std::priority_queue`, побитовая упаковка операциями сдвига и ИЛИ, собственный формат `.huff` с заголовком, таблицей частот и контрольной суммой FNV-1a.

4. Решены практические проблемы: неоднозначность дерева (детерминированный тай-брейк), вырожденный алфавит из одного символа, выравнивание строк `QImage`, защита от повреждённых файлов, быстрая обработка границ окна.
5. Экспериментально подтверждены теоретические положения: $H \leq L_{avg} < H + 1$; коэффициент сжатия побайтового кода Хаффмана не превышает 8 и близок к 1 на данных с высокой энтропией; для малых изображений существенны накладные расходы заголовка.
6. Приложение построено по принципу разделения ответственности: алгоритмические модули не зависят от графического интерфейса, а интерфейс предоставляет две вкладки с полным набором элементов управления и метриками, включая время выполнения в миллисекундах.
7. Исходный код приложения и отчёт опубликованы в репозитории: https://github.com/ArseniyLem/Computer-Graphics/tree/main/lab%202